

Balancing an Inverted Pendulum with a Controlled Flywheel

Siddharth Balraj, Anushka Dharmasanam, Bryant Harrison, Jonah Isaza, and Bowen Lewis
Georgia Institute of Technology, Spring 2026, AE4610 Group A08-B

Reaction wheels are widely used throughout the aerospace industry – from satellites to robotics. Thus, it is important to learn to control a reaction wheel to maintain stability. This experiment involved using a reaction wheel to balance an inverted pendulum to its upright position. The wheel and pendulum were modeled in SolidWorks and 3D-printed, with a 12 volt motor used to spin the pendulum and a counterweight located on the opposite end of the pendulum. The Arduino IDE was used to program responses from an IMU reading both pendulum angle and angular velocity and turn those responses into motor action. This was done by implementing a proportional-integral-derivative controller with gains initially found by performing an linear-quadratic regulator design method.

Then, the controller was implemented and tuned. The final gains through experimental tuning were $K = [5.0 \ 0.5 \ 1.0]$. These values successfully balanced the reaction wheel in the upright position, with unpredictable jolts to either direction (likely due to the high integral gain). The system could take impulse forces – applied by an observer hitting the system in one direction – and would return to the upright position. The data was simulated using SIMULINK and the behavior of the experimental data resembled the simulated data obtained. However, only one of the four controller specifications were actually met. This leaves room for much improvement in future iterations. The easiest change to make would be using a lighter motor, which would reduce the need for a counterweight. Overall, though, the project was successful in maintaining balance using a reaction wheel.

Name	Contribution
Siddharth Balraj	Designed SIMULINK model; assisted with controller implementation; developed counterweight; wrote simulation section.
Anushka Dharmasanam	Assisted with controller implementation, made poster; wrote controller evaluation section.
Bryant Harrison	Modeled system; assisted with controller implementation; wrote abstract, introduction, system dynamics modeling section, and conclusion.
Jonah Isaza	Modeled system; conducted controller design; ran system simulation; assisted with controller implementation; wrote controller design section.
Bowen Lewis	Modeled system; designed in SolidWorks, printed, and built most hardware; wrote hardware implementation section.

AE 4610 Final Project

Synopsis: Rubrics for your project poster and report are included in the following pages. Note that the project is worth 35% of your final grade. An incomplete project due to lack of effort will result in a significant penalty in your project grade.

The weights of the different project components towards the project grade are as follows:

- Proposal: 10% [Graded out of 100 and then reduced to 10]
- Report: 35% [Graded out of 100 and then reduced to 35]
- Poster: 25% [Graded out of 50 and then reduced to 25]
- Peer reviews: 15% [Graded out of 10 and then scaled to 15]
- Successful completion: 15% [Graded out of 15]

In addition, there will be bonus points for the top five poster presentations selected and will be recommended for display on the AE digital board.

Poster Format

- Poster Size: **22 x 28 inches**. Foam boards of same dimensions are available for use as poster boards, and you will clip your poster to these foam boards.
- Title, Group # and Names: Include the title of your project at the top with your section and group number followed by names of students in your group.
- Use any format of your choice for your project poster but make sure that the information (both text and images) is cohesive, clear and without clutter.

Report Format

- Write the report as if the reader has NO prior knowledge about the topic or control system.
- Extra materials such as code, derivation and images should be included in the Appendix.
- Minimum of 5 pages, excluding cover page, plots, tables, appendix. Use the suggested number of pages in the rubric as a guide to limit the number of pages of your report.
- Report should look professionally written, i.e., no formatting error, no hand drawn plots, incomplete equations.
- Use [AIAA](#) manuscript template. [-10 pts if not followed]

Poster Rubric [50 points]

Introduction and Motivation	5 points
- What is the system that is to be controlled? - What real-world applications does this extend to?	
Hardware Setup	10 points
- Include basic details about hardware, including a circuit diagram, a list of measured variables and associated sensors, and actuators/inputs - System modeling/identification - Include images of the setup if not performing live demo	
Controller Design	10 points
- Type of controller chosen and reasons for choosing it - Steps taken to ensure stable closed-loop system, including gain tuning - Quantified desired controller performance criteria (ex. max 5% overshoot, 2 second settling time, etc.)	
Evaluation and Results	10 points
- If successful, include achieved controller design criteria and response graphs with final gains - If not successful yet, include current progress and pending steps	
Conclusion	5 points
- Summary of findings - Highlight future extensions and improvements	
Formatting	10 points
- Cohesive and clear, with proper flow of information and without clutter	

Report Rubric [100 points]

Cover Page	5 points
- Include the title of your project, your section and group number. - List out the names of group members, and their individual contributions to the project. - Specifics required! Simply listing the section or task would not be accepted. [-5 pts if not followed] E.g.: Person A: Developed the equations of motion, performed sensor and actuator calibration, machined the base plate and other components, identified controller specifications, wrote the system ID section of the report. Person B: Designed the experimental setup on CAD, performed the system ID experiments to identify parameters X and Y, developed Simulink model and performed simulations, wrote the controller design section of the report. Person C: Person D:	
Summary (~1 page)	5 points
Highlight the motivation for the project, the overall objective(s) and the overall success of the project in 1-2 paragraphs.	
Introduction (~1 page)	5 points
- Brief background with references about the topic [3 pts] - Brief description of experimental set up [2 pts]	
System Dynamics Modeling (~2 pages)	10 points
- Study of the system being controlled and justification of modeling [4 pts] - Mathematical model of the open loop system [1 pt] - Assumptions and approximations [1 pt] - System identification process and results (include a brief description of the experiments conducted and data collected) [4 pts]	
Controller Design (~2 pages)	10 points
- Controller design specifications [2.5 pts] - Type of controller used and a block diagram of the controller [2.5 pts] - Controller design steps and results (time-domain, frequency-domain, root locus, pole-zero map, etc.) [5 pts]	
Simulation (~2 pages)	15 points
- Simulink diagrams or other simulation software used [5 pts] - Controller tuning, how changes affect the response [5 pts] - Show some results of your gain tuning iterations - Response plots, check if specifications are met [5 pts]	

Hardware Implementation and Controller Evaluation (~5 pages)	40 points
- Table of components, specifications if necessary [5 pts] - Pictures of hardware set-up, wiring schematics, etc. [5 pts] - Design/Fabrication/Manufacturing and sensor/actuator testing and calibration [10 pts] [20 pts for either of the two below] - Final controller tuning and results (including response plots) and proposed future extensions, along with live demo during poster presentation or a link to recorded video. - If controller evaluation is unsuccessful, explain in detail how and why it failed. Include plots/figures to support explanation. Provide feasible solution (in detail) to overcome the problems and pending future work.	
Conclusion (~1 page)	5 points
- Summary of the experiments [2 pts] - Key findings from the results [3 pts] - NO new additional results/findings/calculations [-1 pt for this]	
References (~0.5 page)	5 points
- Proper reference citation within the report [3 pts] - Relevant reference [2 pts]	

I. Nomenclature

θ	=	Angle of inverted pendulum measured from upright vertical position
$\dot{\theta}$	=	Angular velocity of the inverted pendulum
ϕ	=	Angle of the reaction wheel relative to the inverted pendulum
$\dot{\phi}$	=	Angular speed of the reaction wheel relative to the pendulum
ω_w	=	Angular speed of the system
τ	=	Motor torque applied to the reaction wheel
m	=	Mass of the system
l	=	Distance from the pivot point to the center of mass of the system
J_p	=	Moment of inertia of the inverted pendulum about the pivot
J_w	=	Moment of inertia of the reaction wheel about its spin axis
T	=	Kinetic energy
V	=	Potential energy
g	=	Gravitational constant on Earth's surface
L	=	Lagrangian
q	=	Generalized coordinates
Q	=	Generalized forces
x	=	State-space variables
$\dot{x}$	=	Derivatives of the state-space variables
u	=	State-space input
A, B, C, D	=	Matrices associated with the state-space
y	=	State-space output
K	=	Gain matrix
k	=	Controller gains
t	=	Time elapsed

II. Introduction

THE purpose of this project is to design a self-balancing inverted pendulum via a reaction wheel. An inverted pendulum on its own represents an unstable equilibrium point. That is, it can maintain upright as long as no forces interact with it, as any force will knock it off of the equilibrium point to swing down. However, sometimes this unstable point is needed to be maintained for system functionality. The main body, an arm that can rotate freely within its stand, is attached to a flywheel which stabilizes the arm about its axis of rotation. The stand is stabilized by the flywheel by rotating a motor connected to the flywheel opposite the direction of motion. Once stabilized, this stand should be able to also take impacts and return to an upright position. The design of the system in this manner is then open-loop unstable, like an inverted pendulum. The addition of a controller can stabilize the system.

The physical components used for this project are all 3D printed. This includes the arm and the flywheel. The 3D printed parts are screwed into a wooden board to serve as the base of the system. The controller electronics include the 200RPM/12V DC motor, an Arduino Uno Rev3 SMD and motor shield, an H-bridge, an Adafruit 9-DOF IMU, and a 12V power supply. The most important circuit component is the IMU, designed to read the arm angle and angular velocity. These values are used to control the motor. The arm is placed on a socket on the stand within which it can freely rotate. This socket is comprised of ball bearings with the intention of mitigating any rotational friction present. The reaction wheel is at the free end of arm, attached to a motor. The electronics are connected to this motor. More details about the setup are shown in the hardware implementation section.

The inspiration behind this project was a flywheel stabilization system demonstrated by ReM-RC [1]. Though this stand is comprised of two flywheels stabilizing a stand with only one contact point, this project alters it to be a simpler, one degree-of-freedom system. The simplifications are discussed in detail in the system dynamics modeling section, but the key assumption is the linearization about the upright position when deriving the equations of motion of the system. The most important concept behind a reaction wheel is conservation of angular momentum. The motor introduces a counteractive angular momentum that sets the net momentum to zero (theoretically). A reaction wheel has several applications in the field of aerospace engineering and beyond. Most prominently, satellites use reaction wheels to maintain its attitude without external thrusters. Similarly, reaction wheels could balance a robotic limb for a wide variety of purposes, such as maintaining stability for a camera shot. The success of this project can demonstrate the method in

which balancing stability can be obtained for real-world systems. This is valuable to both the the authors' learning as engineers and the wider field of engineering as a whole. The following sections go into detail on the modeling, controller design, simulation, hardware implementation, and controller evaluation of the inverted pendulum with a reaction wheel.

III. System Dynamics Modeling

A. Modeling Assumptions

The following assumptions were used in modeling:

- 1) Motion is restricted to a single plane.
- 2) The system can be modeled as a single degree of freedom system.
- 3) The inverted pendulum is rigid.
- 4) The reaction wheel is rigid and spins about a fixed axis on its attachment to the inverted pendulum.
- 5) "Motor torque" refers to the system input.
- 6) Balancing occurs around the vertical position, so the small angle approximation can be used.
- 7) All other forces and considerations (drag, friction, etc.) are ignored.

These assumptions were critical in simplifying the system to be controlled and stabilized in the upright position. The system dynamics are to be developed using the diagram in Figure 1. In subsequent discussion, a subscript of p refers to

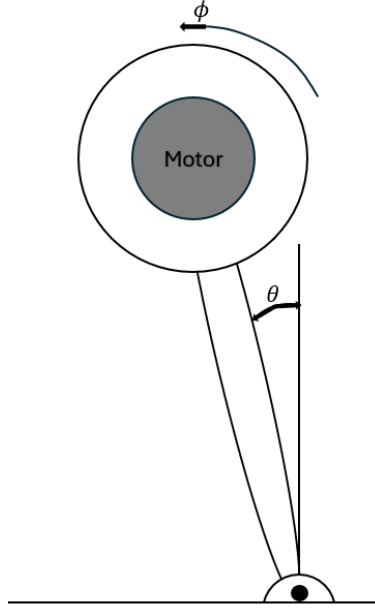


Fig. 1 Simplified model of the system, a pendulum with a reaction wheel.

the pendulum arm, and a subscript of w refers to the reaction wheel.

B. Equation of Motion

The following sections describe the derivation of the system equation of motion (EOM).

1. Kinetic Energy

The total kinetic energy of the system is the sum of the kinetic energy of the pendulum and the kinetic energy of the reaction wheel. The inverted pendulum has moment of inertia J_p and rotates with speed $\dot{\theta}$, so its kinetic energy is:

$$T_p = \frac{1}{2} J_p \dot{\theta}^2. \quad (1)$$

The reaction wheel has moment of inertia J_w and rotates with the absolute angular speed $\dot{\theta} + \dot{\phi}$. Thus, the kinetic energy for the wheel is:

$$T_w = \frac{1}{2} J_w (\dot{\theta} + \dot{\phi})^2. \quad (2)$$

Summing these two kinetic energies in Equations (1) and (2) yields the total kinetic energy of the system:

$$T = T_p + T_w \quad (3a)$$

$$= \left[\frac{1}{2} J_p \dot{\theta}^2 \right] + \left[\frac{1}{2} J_w (\dot{\theta} + \dot{\phi})^2 \right] \quad (3b)$$

$$= \frac{1}{2} J_p \dot{\theta}^2 + \frac{1}{2} J_w \dot{\theta}^2 + J_w \dot{\theta} \dot{\phi} + \frac{1}{2} J_w \dot{\phi}^2 \quad (3c)$$

$$T = \frac{1}{2} (J_p + J_w) \dot{\theta}^2 + J_w \dot{\theta} \dot{\phi} + \frac{1}{2} J_w \dot{\phi}^2. \quad (3d)$$

2. Potential Energy

The only potential energy in this system is the gravitational potential energy due to the mass of the system. That is,

$$V = mgl \cos \theta. \quad (4)$$

3. Lagrangian

The Lagrangian is defined as

$$L = T - V. \quad (5)$$

Substituting Equations (3d) and (4) into Equation (5) yields:

$$L = \left[\frac{1}{2} (J_p + J_w) \dot{\theta}^2 + J_w \dot{\theta} \dot{\phi} + \frac{1}{2} J_w \dot{\phi}^2 \right] - [mgl \cos \theta] \quad (6a)$$

$$L = \frac{1}{2} (J_p + J_w) \dot{\theta}^2 + J_w \dot{\theta} \dot{\phi} + \frac{1}{2} J_w \dot{\phi}^2 - mgl \cos \theta. \quad (6b)$$

4. Generalized Forces

The motor applies an internal torque to the reaction wheel that then affects the stick body. Thus, the generalized forces are

$$Q_\theta = 0 \quad (7)$$

and

$$Q_\phi = \tau. \quad (8)$$

5. Euler-Lagrange Equations

The generalized coordinates are selected to be the two angles in the system, that is $q_1 = \theta$ and $q_2 = \phi$. Then, for each generalized coordinate, the Euler-Lagrange equation is

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = Q_i \quad (9)$$

For q_1 :

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) - \frac{\partial L}{\partial \theta} = Q_\theta \quad (10a)$$

$$\frac{d}{dt} ([J_p + J_w] \dot{\theta} + J_w \dot{\phi}) - (mgl \sin \theta) = 0 \quad (10b)$$

$$(J_p + J_w) \ddot{\theta} + J_w \ddot{\phi} - mgl \sin \theta = 0. \quad (10c)$$

For q_2 :

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\phi}} \right) - \frac{\partial L}{\partial \phi} = Q_\phi \quad (11a)$$

$$\frac{d}{dt} (J_w \dot{\theta} + J_w \dot{\phi}) - (0) = \tau \quad (11b)$$

$$J_w \ddot{\theta} + J_w \ddot{\phi} = \tau. \quad (11c)$$

Rearranging Equation (11c) yields:

$$J_w \ddot{\phi} = \tau - J_w \ddot{\theta}, \quad (12)$$

which can be plugged into Equation (10c) to find that

$$(J_p + J_w) \ddot{\theta} + (\tau - J_w \ddot{\theta}) - mgl \sin \theta = 0 \quad (13a)$$

$$J_p \ddot{\theta} - mgl \sin \theta = -\tau. \quad (13b)$$

Then, applying the small angle approximation yields the final form for the EOM:

$$J_p \ddot{\theta} - mgl \theta = -\tau. \quad (14)$$

Since the motor applies torque directly to the reaction wheel, a second "EOM" can be generated based on rotational mechanics. This equation is

$$\dot{\omega}_w = \frac{\tau}{J_w}. \quad (15)$$

6. Conversion to State Space

The state space form is the following:

$$\dot{x} = Ax + Bu \quad (16)$$

$$y = Cx + Du. \quad (17)$$

The state variables are then chosen to be

$$x_1 = \theta, \quad (18)$$

$$x_2 = \dot{\theta}, \quad (19)$$

and

$$x_3 = \omega_w. \quad (20)$$

Then, the input to the system is the motor torque. That is,

$$u = \tau \quad (21)$$

The derivatives of the state variables are then:

$$\dot{x}_1 = \dot{\theta} \quad (22a)$$

$$= x_2, \quad (22b)$$

$$\dot{x}_2 = \ddot{\theta} \quad (23a)$$

$$= \frac{mgl\theta}{J_p} - \frac{\tau}{J_p} \quad (23b)$$

$$= \frac{mglx_1}{J_p} - \frac{u}{J_p}, \quad (23c)$$

and

$$\dot{x}_3 = \dot{\omega}_w \quad (24a)$$

$$= \frac{\tau}{J_w} \quad (24b)$$

$$= \frac{u}{J_w}. \quad (24c)$$

As a vector, the state derivative is

$$\dot{x} = \begin{bmatrix} x_2 \\ \frac{mglx_1}{J_p} - \frac{u}{J_p} \\ \frac{u}{J_w} \end{bmatrix}. \quad (25)$$

Then, the state space matrices follow from Equation (25) to be:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ \frac{mgl}{J_p} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (26)$$

and

$$B = \begin{bmatrix} 0 \\ \frac{-1}{J_p} \\ \frac{1}{J_w} \end{bmatrix} \quad (27)$$

The other two matrices are selected to be:

$$C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \quad (28)$$

and

$$D = 0. \quad (29)$$

Thus, the finalized state space equations are:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ \frac{mgl}{J_p} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{-1}{J_p} \\ \frac{1}{J_w} \end{bmatrix} u \quad (30)$$

and

$$y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}. \quad (31)$$

C. System Identification Process

No system identification was conducted for this experiment. Instead, the parameters of the system were measured directly from the design created. The input values for the motor torque were bypassed by using the angular velocity measured by the system IMU. This allowed for the project to continue without determining the specific motor parameters and greatly simplified the controller design process. The system parameters were found to be the following values:

$$m = 0.23746 \text{ kg} \quad (32a)$$

$$l = 0.153 \text{ m} \quad (32b)$$

$$J_p = 0.004514 \text{ kgm}^2 \quad (32c)$$

$$J_w = 4.73315 * 10^{-4} \text{ kgm}^2. \quad (32d)$$

Thus, the state-space can be found to numerically be

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 79.0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ -221.5 \\ 2112.76 \end{bmatrix} u, \quad (33)$$

along with Equation (31) which remains unchanged.

IV. Controller Design

A. Controller Specifications

1. Controller Requirements

The following controller specifications were decided upon by the authors in the proposal:

- 1) Rise time, $t_r \leq 1$ s
- 2) Settling time, $t_s \leq 2$ s ($\pm 5\%$)
- 3) Steady state error, $e_{ss} = 3^\circ$
- 4) Overshoot, $OS = 13^\circ$ for θ

To design a suitable controller, the linear quadratic regulator (LQR) method was implemented. This method uses chosen maximum excursion values for the three states along with a maximum control effort value to pick a vector of gains, K . For this method, these values were picked as follows:

- $\theta_{max} = .052$ rad
- $\dot{\theta}_{max} = .1 \frac{\text{rad}}{\text{s}}$
- $\omega_{max} = 2\pi \frac{\text{rad}}{\text{s}}$
- $U_{max} = 5$ V

The values pertaining to the motor performance, ω_{max} and τ_{max} were chosen based off of the data sheet for the motor used. The maximum excursion of the arm, θ was chosen based off of the requirements enumerated above. The value of $\dot{\theta}$ was not based on any particular data source, being chosen by engineering judgment on behalf of the authors.

B. Controller Theory

This controller implements a full-state feedback controller, where the states are defined as:

- 1) θ , Position of the pendulum, rad
- 2) $\dot{\theta}$, Angular velocity of the pendulum, $\frac{\text{rad}}{\text{s}}$
- 3) ω , Angular velocity of the wheel, $\frac{\text{rad}}{\text{s}}$

Using the modeling derivations in subsubsection III.B.6, a full-state controller can be designed around these states. This controller takes the form of a gain vector K , where:

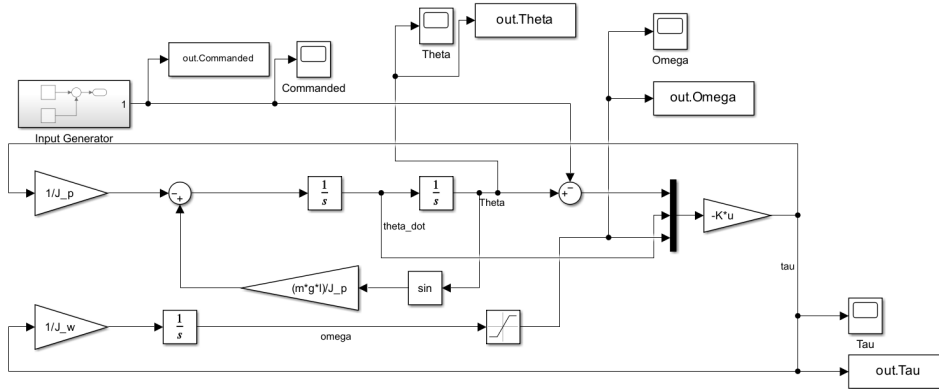
$$K = \begin{bmatrix} K_1 & K_2 & K_3 \end{bmatrix} \quad (34)$$

The controller law takes the form of:

$$u = -Kx = \begin{bmatrix} K_1 & K_2 & K_3 \end{bmatrix} \begin{bmatrix} \theta \\ \dot{\theta} \\ \omega \end{bmatrix} \quad (35)$$

The implementation of this control law into a model of this system was done using SIMULINK, and is displayed in

Figure 2



The mechanism behind the LQR method for the selection of the gain vector K is complex, but ultimately relies upon the system being controllable. A matrix, T , can be constructed from the state-space of the system and if the rank of this matrix is equal to the states of the system (3), then the system is deemed controllable. By using the fact that this system is controllable (displayed in Equation (37)), the LQR method can be used. This method solves some cost function. Equation (38), with two matrices Q and R that are defined based on the excursions chosen in subsubsection IV.A.1. These matrices are chosen based on Byrson's rule, a common method used in LQR controller design.

$$\text{rank}(T) = 3 \tag{37}$$

$$Q = \begin{bmatrix} \frac{1}{\theta_{max}^2} & 0 & 0 \\ 0 & \frac{1}{\theta_{max}^2} & 0 \\ 0 & 0 & \frac{1}{\omega_{max}^2} \end{bmatrix} \quad (39)$$

The LQR method shown here can be implemented manually, but was implemented in MATLAB using the `lqr()` command.

After using the linear-quadratic regulator (LQR) method of controller design, the following gain vector was obtained.

The root-locus of this controlled system is displayed in Figure 3. All poles of the system are located in the left-hand plane, proving closed-loop stability.

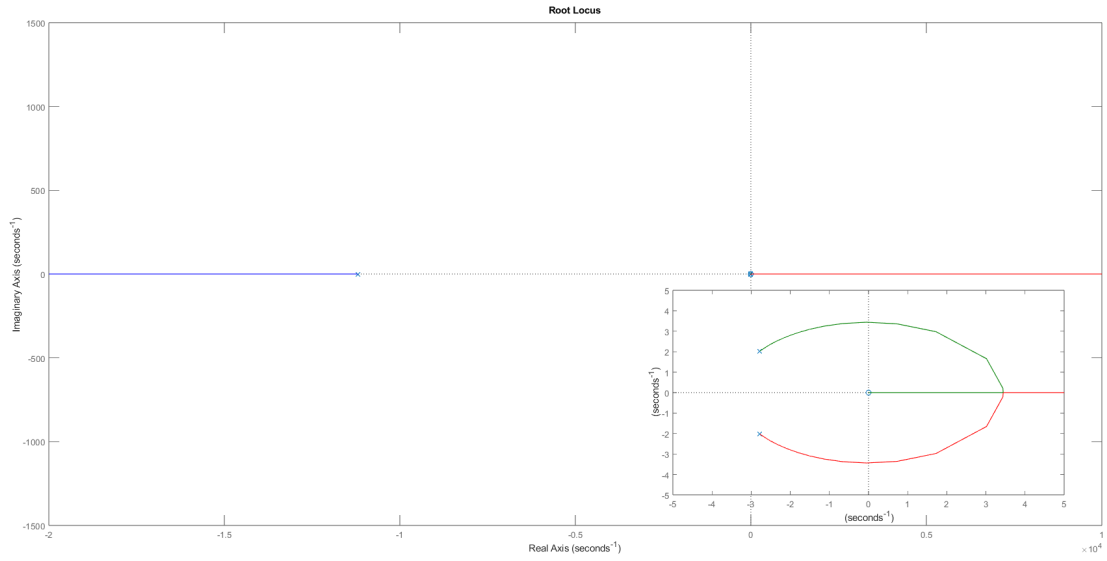


Fig. 3 Root-Locus of the system with the control law implemented

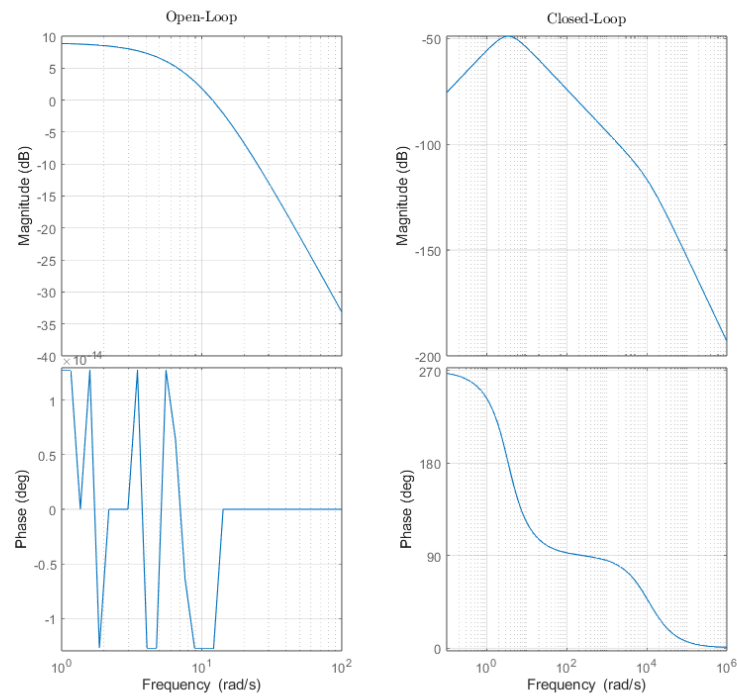


Fig. 4 Bode plots of the open and closed-loop system

A time-domain response of the system is shown in Figure 5. It can be seen from this time-domain response that the system stabilizes within the requirements, and thus the controller is deemed to have met initial requirements.

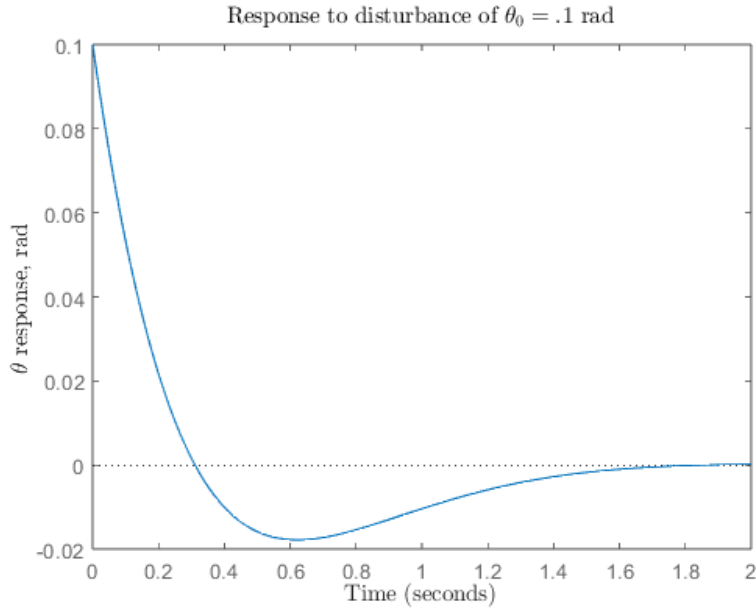


Fig. 5 Step response of the system

V. Simulation

A. Simulink Model

The Simulink model used to simulate the system is shown in Figure 6. The Simulink model implements the full nonlinear equations of motion from the previous sections using a sine block for the gravitational term. The input generator applies a step command of -0.1 rad at $t = 1$ s followed by a step of $+0.1$ rad at $t = 2$ s, resulting in a net commanded displacement of -0.1 rad that returns to zero. The state vector (arm angle θ , angular rate $\dot{\theta}$, and wheel velocity ω) is fed back through the gain matrix K computed in MATLAB. The control torque τ passes through a saturation block limiting wheel velocity to $\pm 2\pi$ rad/s. The arm angle, control torque, and wheel velocity are logged as outputs from the simulation.

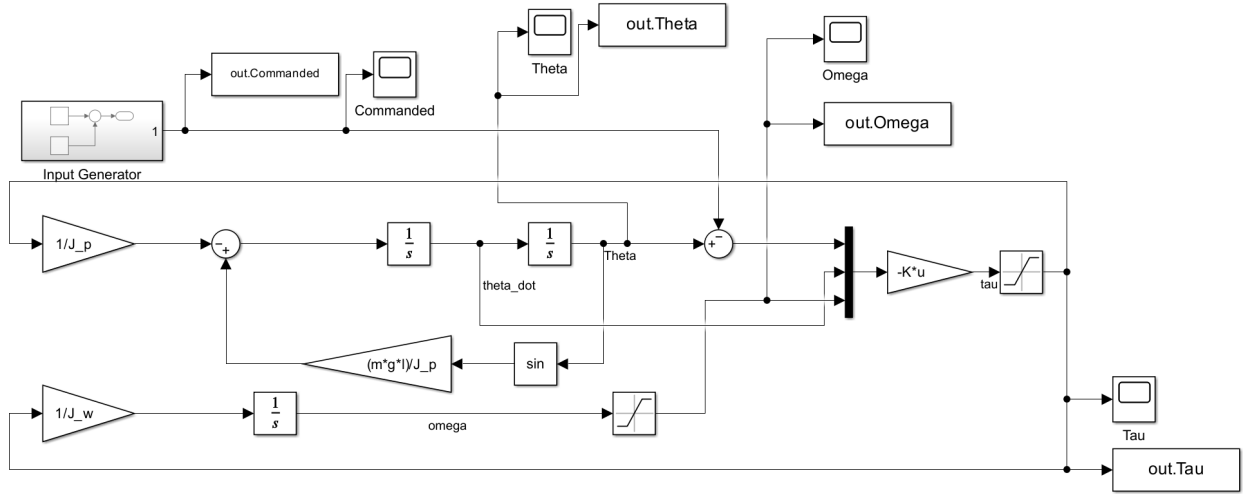


Fig. 6 Flywheel Inverted Pendulum Simulink Model.

B. Baseline LQR Gains

The baseline simulation used the LQR gains from Section IV, computed using Bryson's rule with the maximum excursion values of $\theta_{max} = 0.052$ rad, $\dot{\theta}_{max} = 0.1$ rad/s, $\omega_{max} = 2\pi$ rad/s, and $U_{max} = 5$:

$$K = \begin{bmatrix} -281.39 & -58.19 & -0.796 \end{bmatrix} \quad (42)$$

Plots of arm angle, control torque, and wheel velocity response for the baseline gains are shown in Figures 7, 8, and 9.

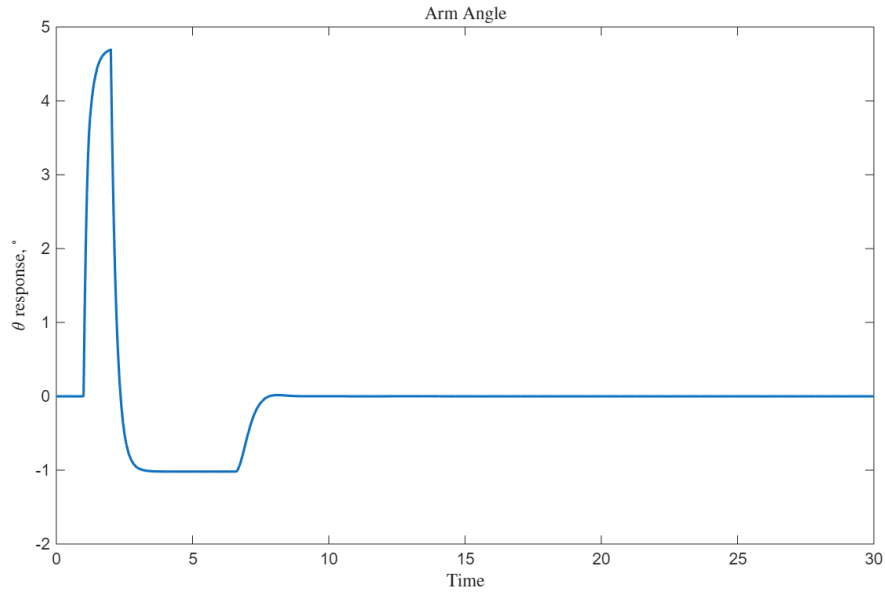


Fig. 7 Baseline arm angle response.

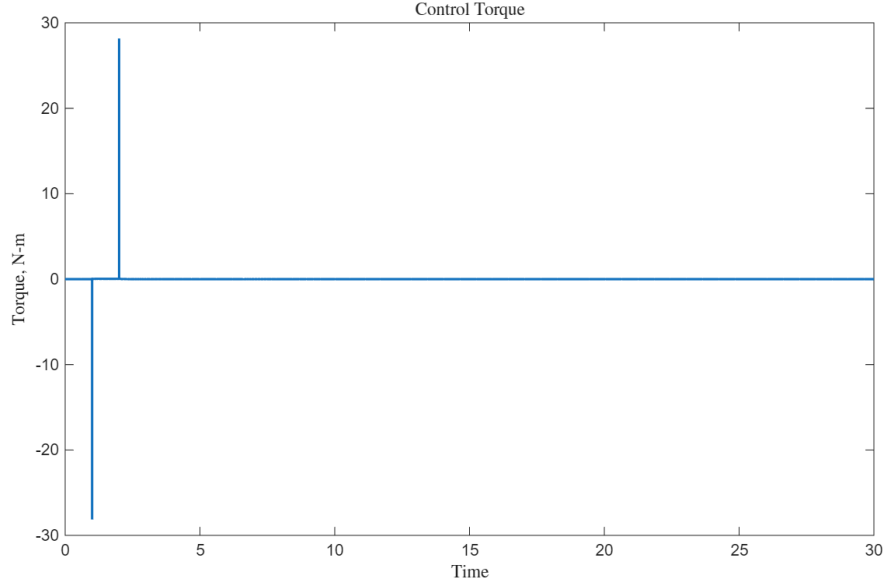


Fig. 8 Baseline control torque.

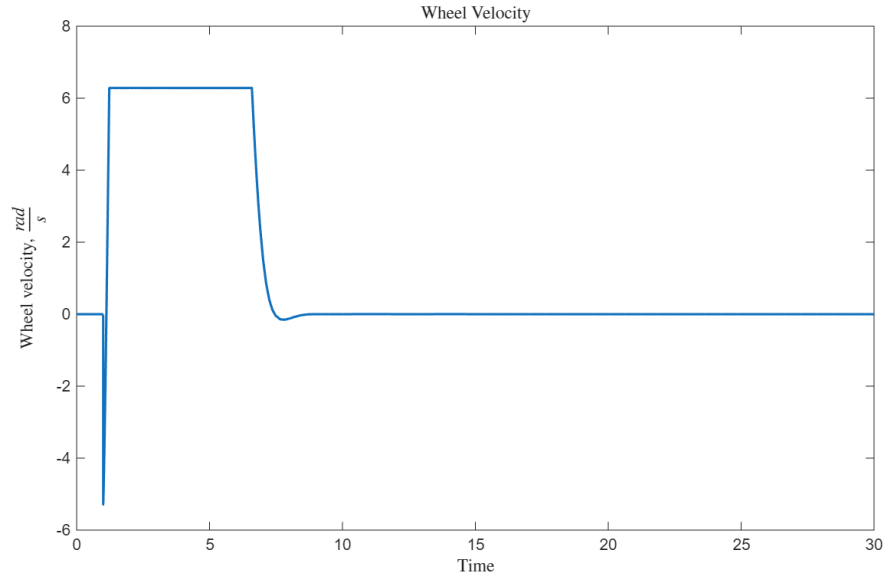


Fig. 9 Baseline wheel velocity.

The arm angle peaks at approximately 4.8° around $t = 2$ s and settles to near zero by $t \approx 8$ s. The rise time under the 1-second requirement is satisfied, but the settling time of 8 seconds is above the 2-second requirement. Overshoot of 4.8° and near-zero steady-state error both meet their respective specifications.

The torque spikes to ± 28 N-m during the transient before dropping to zero once the arm settles. This is because the Simulink model limits only the wheel velocity through saturation and leaves the torque unconstrained. The ± 28 N-m peak likely exceeds the physical capability of the motor used in hardware.

The wheel velocity saturates at $\pm 2\pi$ rad/s almost immediately, then unwinds back to near zero as the arm settles. Since the wheel is held at its limit during most of the transient phase, the controller likely can not act dynamically during that period, and could be a reason for the long settling time.

C. Gain Tuning Iterations

Two additional cases were simulated by modifying the Bryson's rule parameters to observe their effect on system response.

1. Iteration 1: Tighter Angle Constraint ($\theta_{max} = 0.02$ rad)

Reducing θ_{max} from 0.052 rad to 0.02 rad increases the (1,1) entry of Q , which penalizes angular deviation more heavily and produces more aggressive gains:

$$K = \begin{bmatrix} -364.009 & -58.195 & -0.796 \end{bmatrix} \quad (43)$$

Plots of arm angle, control torque, and wheel velocity response for the iteration 1 gains are shown in Figures 10, 11, and 12.

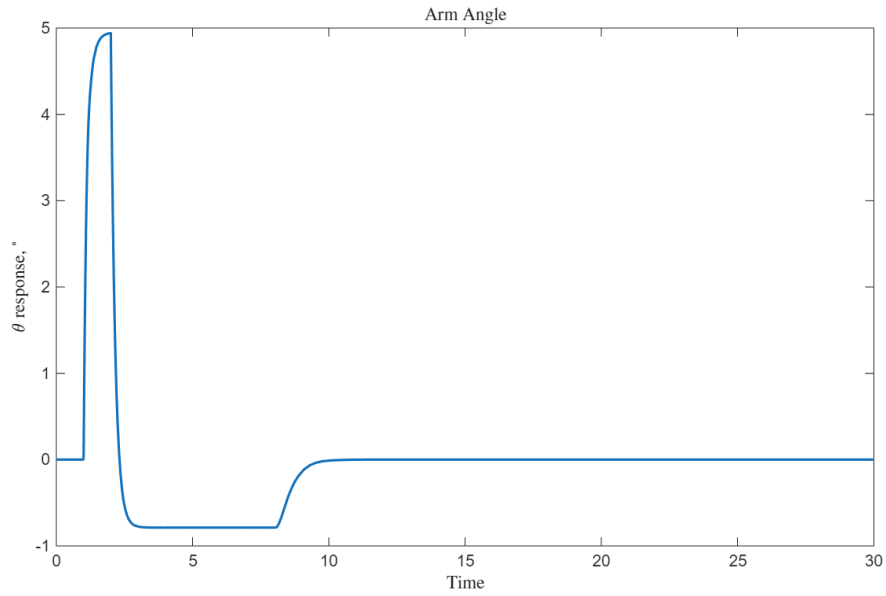


Fig. 10 Arm angle response with tightened angle constraint ($\theta_{max} = 0.02$ rad).

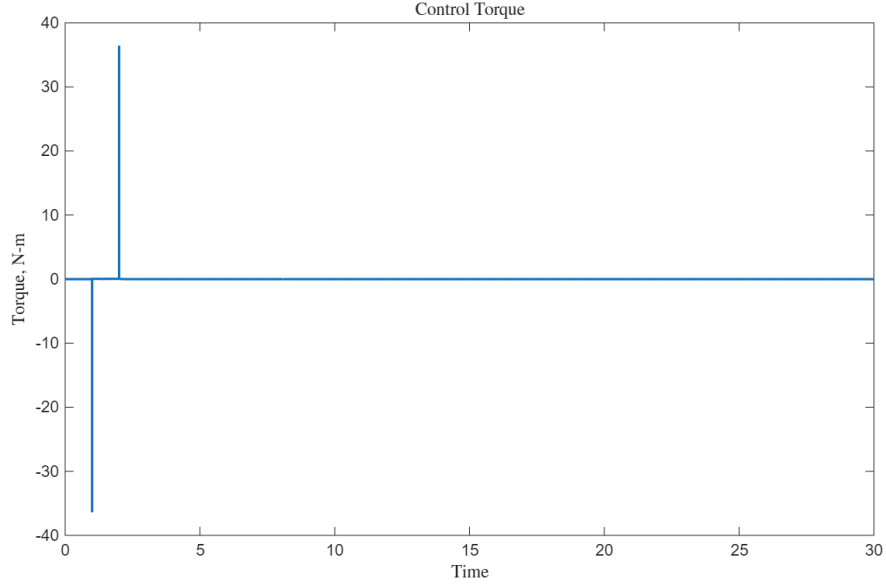


Fig. 11 Control torque with tightened angle constraint ($\theta_{max} = 0.02$ rad).

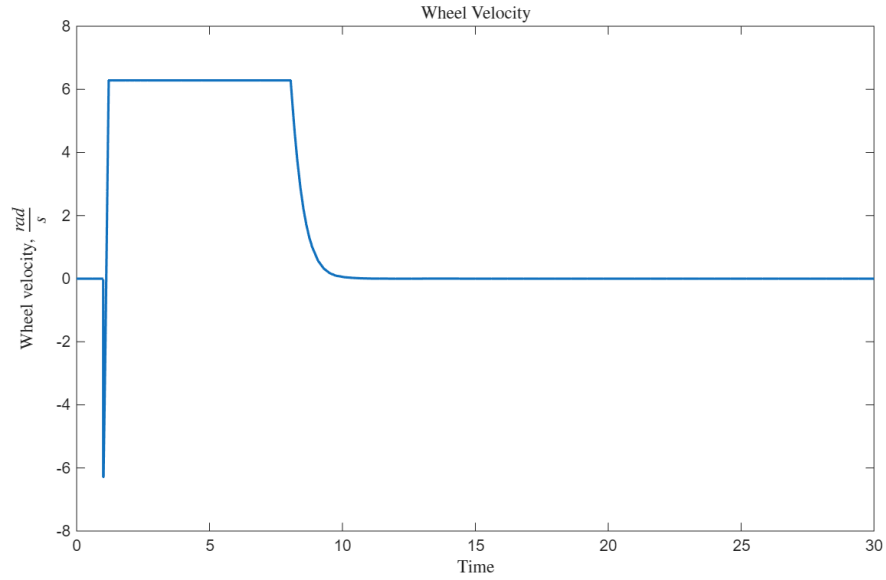


Fig. 12 Wheel velocity with tightened angle constraint ($\theta_{max} = 0.02$ rad).

Despite the larger gains, the arm angle response is nearly identical to the baseline, peaking at 4.9° and settling by $t \approx 10$ s. The torque demand increases to ± 36 N-m. The similarity in angular response is explained by the wheel saturating quickly again, so the additional control authority has less effect. This shows that beyond a certain threshold, increasing the state penalty in Q yields no improvement when the actuator is already saturated.

2. Iteration 2: Higher Input Penalty ($U_{max} = 1$)

Reducing U_{max} from 5 to 1 increases R , which penalizes control effort more heavily and produces smaller gains:

$$K = \begin{bmatrix} -56.607 & -11.658 & -0.159 \end{bmatrix} \quad (44)$$

Plots of arm angle, control torque, and wheel velocity response for the iteration 2 gains are shown in Figures 13, 14, and 15.

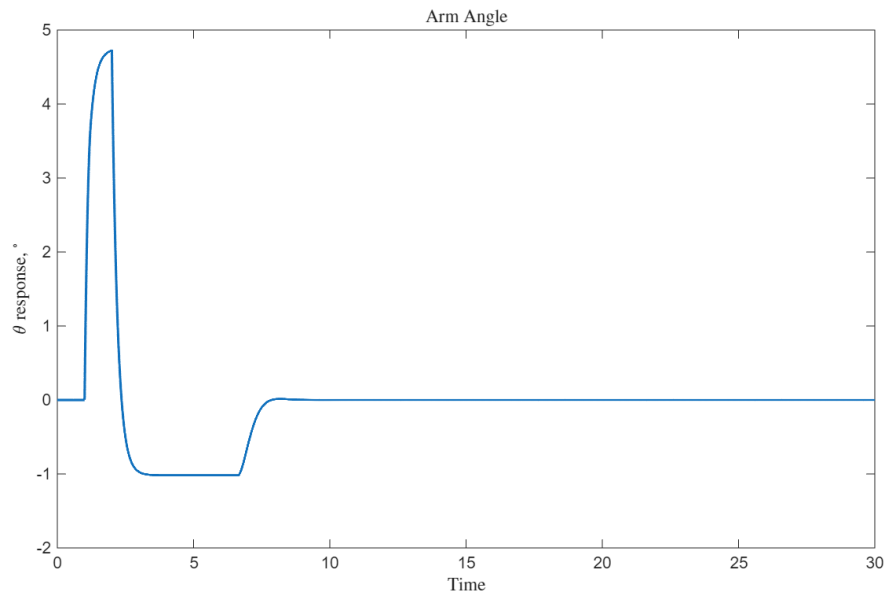


Fig. 13 Arm angle response with higher input penalty ($U_{max} = 1$).

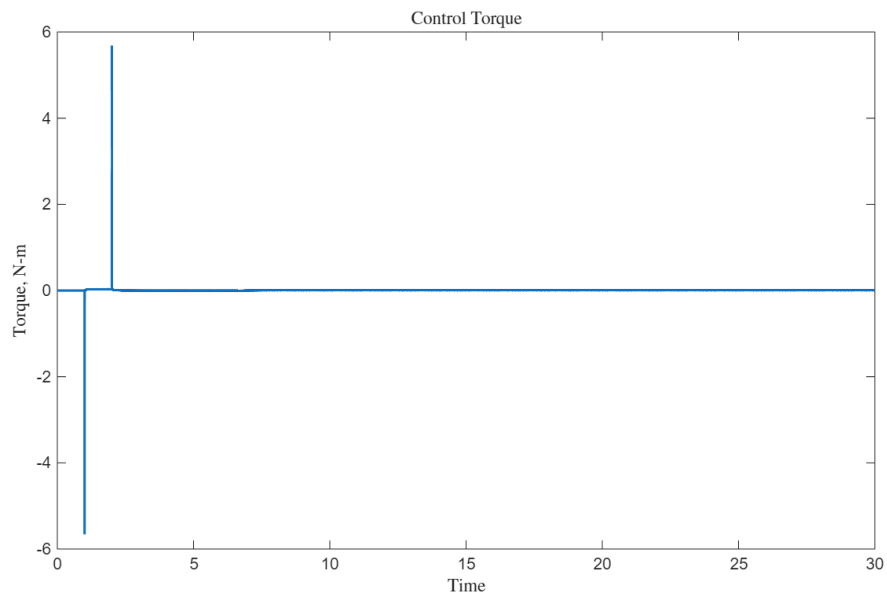


Fig. 14 Control torque with higher input penalty ($U_{max} = 1$).

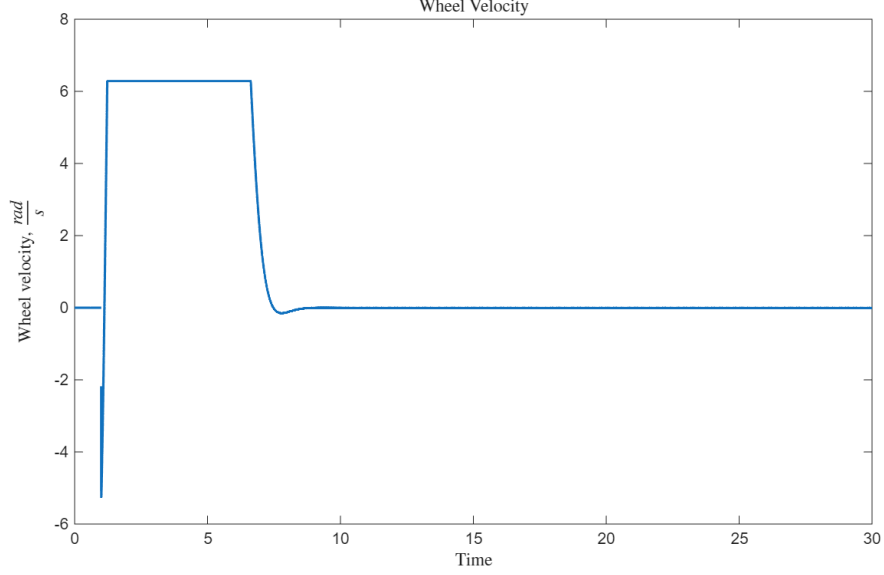


Fig. 15 Wheel velocity with higher input penalty ($U_{max} = 1$).

The arm angle peaks at 4.7° and settles at $t \approx 8$ s, which is slightly faster than the baseline. The most significant result is the torque, which drops to ± 5.7 N-m, indicating around an 80% reduction from the baseline. This is a much more physically realistic value for the motor used in hardware. The wheel velocity plot is similar to the baseline, but settles faster. This iteration shows the best result of the three cases and suggests that higher input penalty may be more necessary to control the motor torque output.

D. Summary of Simulation Results

Table 2 compares the three cases against the controller specifications from Section IV.

Table 2 Simulation results compared against controller specifications.

Parameter	Specification	Baseline	Iteration 1 ($\theta_{max} = 0.02$ rad)	Iteration 2 ($U_{max} = 1$)
Rise time, t_r	≤ 1 s	< 1 s	< 1 s	< 1 s
Settling time, t_s	≤ 2 s	~ 8 s	~ 10 s	~ 8 s
Steady-state error, e_{ss}	$\leq 3^\circ$	$\sim 0^\circ$	$\sim 0^\circ$	$\sim 0^\circ$
Overshoot	$\leq 13^\circ$	4.8°	4.9°	4.7°
Peak torque	—	± 28 N-m	± 36 N-m	± 5.7 N-m

Rise time, steady-state error, and overshoot are met across all three cases. Settling time is not met in any case. The gain tuning results show that increasing the angle penalty in Q has little effect on the angular response when the actuator is already saturated, while increasing the input penalty in R greatly reduces torque demand. This reduction in torque output also comes with very few changes in the angle response. The failure to meet the settling time specification across all iterations points to the wheel velocity saturation as the main cause.

VI. Hardware Implementation and Controller Evaluation

A. Hardware Design

The final hardware implementation of the reaction-wheel inverted pendulum was developed as a 3D printed experimental platform intended to validate the controller in physical hardware. The final build used an Arduino Uno

R3 as the microcontroller, an Adeeep L298N dual H-bridge motor driver, a Greartisan 12 V geared DC motor, an Adafruit BNO085 9-DOF IMU, and a 12 V battery pack as the power source. Although an AS5048A magnetic encoder was initially considered for measuring wheel speed, it was not used in the final implementation due to integration difficulties and limited time for debugging. The final system therefore relied on the IMU to provide pendulum angle and angular velocity for feedback. The Uno R3 was selected because it provided sufficient digital I/O, PWM capability, and simple integration with the Arduino IDE for control prototyping. The BNO085 was selected because it integrates an accelerometer, gyroscope, magnetometer, and onboard sensor fusion, and supports angular velocity and orientation outputs over standard host interfaces such as I²C, SPI, and UART. The L298N board was used as a simple bidirectional motor driver capable of driving DC motors from a 5–35 V supply with logic-level control inputs.

Component Summary

Table 3 summarizes the major hardware components used in the final build.

Table 3 Final hardware components used in the reaction-wheel balancing system.

Component	Model / Part	Relevant Specifications
Microcontroller	Arduino Uno R3	ATmega328P-based board; up to 16 MHz operation; six PWM channels; USB-B programming and power interface.
Motor Driver	Adeeep L298N Dual H-Bridge	DC motor supply range 5–35 V; peak current 2 A; bidirectional control; control logic high recognized above 2.3 V.
Drive Motor	Greartisan DC Geared Motor	Rated voltage 12 V; reduction ratio 1:3; no-load speed 1000 RPM; rated torque 0.15 kgf-cm; rated current 0.35 A; motor mass 184 g.
IMU	Adafruit BNO085 9-DOF IMU	Integrated accelerometer, gyroscope, and magnetometer; onboard sensor fusion; provides angular velocity and orientation outputs.
Battery	12 V battery pack	Nominal voltage 12 V; capacity 2500 mAh.
Encoder (not used in final build)	AS5048A Adapter Board	14-bit magnetic rotary encoder; SPI/PWM output; 0.0219°/LSB nominal resolution; supports 3.3 V or 5 V operation.
Base Plate	Plywood board	Approximate footprint 1 ft × 2 ft; used as structural mounting platform.
Custom Pivot/Base	Team-designed, 3D printed	Housed skateboard ball bearings to reduce rotational friction.
Pendulum Arm	Team-designed, 3D printed	Single-axis arm with 160 mm length above pivot and 100 mm counterweight arm below pivot.
Reaction Wheel	Team-designed, 3D printed	200 mm diameter wheel with perimeter bolts for added rim inertia; mass approximately 90 g.
Sensor Bracket	Team-designed, 3D printed	Used to mount the IMU and originally intended to also house the encoder.

Mechanical Design and Fabrication

All custom structural components were designed in SolidWorks and manufactured using a Bambu Labs P2S 3D printer. The printed components included the pivot base, pendulum arm, reaction wheel, and a sensor mounting bracket for the IMU and encoder. A plywood board approximately 1 ft by 2 ft served as the ground base for the experiment and provided a surface for mounting the printed support structure, electronics, and wiring. The reaction wheel was mounted at the upper end of the pendulum arm, while a counterweight arm extended below the pivot in order to offset the large mass of the DC motor mounted near the top of the system. The final arm length above the pivot was 160 mm, and the

lower counterweight arm extended 100 mm below the pivot. The reaction wheel diameter was 200 mm and its mass was approximately 90 g. These dimensions were chosen to keep the overall structure compact while still providing enough flywheel radius to generate appreciable reaction torque.

The first iterations of the mechanical design had too much friction at the pivot, which severely limited the ability of the reaction wheel to influence the pendulum motion. To reduce this friction, the pivot was redesigned to use skateboard ball bearings. During post-processing, the ball bearings were super-glued into the printed base and the pendulum arm was bonded to the bearing-supported pivot so that the arm could rotate freely in a single plane. This redesign represented one of the most important hardware improvements of the project, since reducing pivot friction significantly improved the responsiveness of the arm to reaction torques. The final design intentionally constrained the pendulum to one degree of freedom so that the hardware matched the assumptions used in the system modeling section.

The printed parts were manufactured with an infill of 15%, which was selected to reduce print time and mass while still maintaining sufficient rigidity for testing. This decision was important for the pendulum arm and flywheel, where unnecessary mass would increase the moment of inertia and make stabilization more difficult. Although the printed reaction wheel provided the geometric form of the flywheel, bolts were added around the outer rim to shift more mass outward and increase the effective rotational inertia. This approach allowed the wheel to behave more like a true reaction wheel while still using a lightweight printed structure.

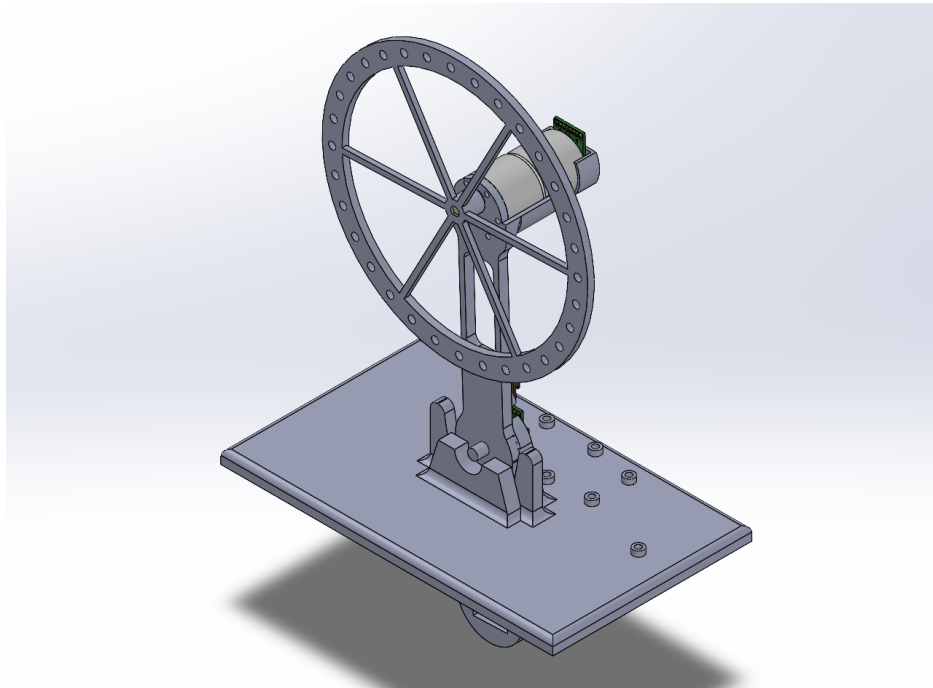


Fig. 16 CAD model of the hardware assembly.

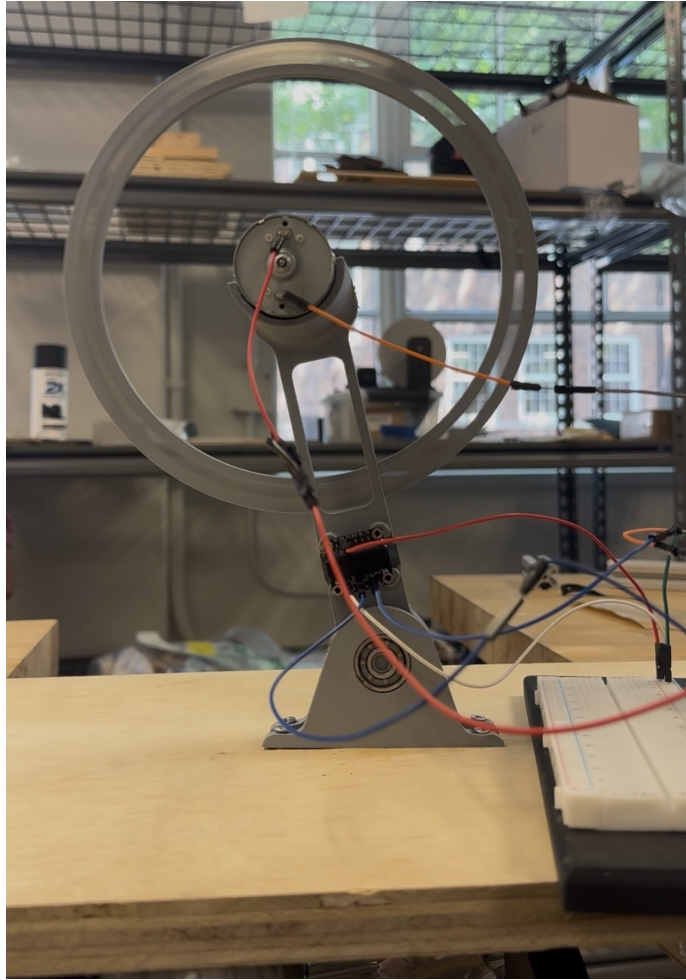


Fig. 17 Final Assembled Hardware.

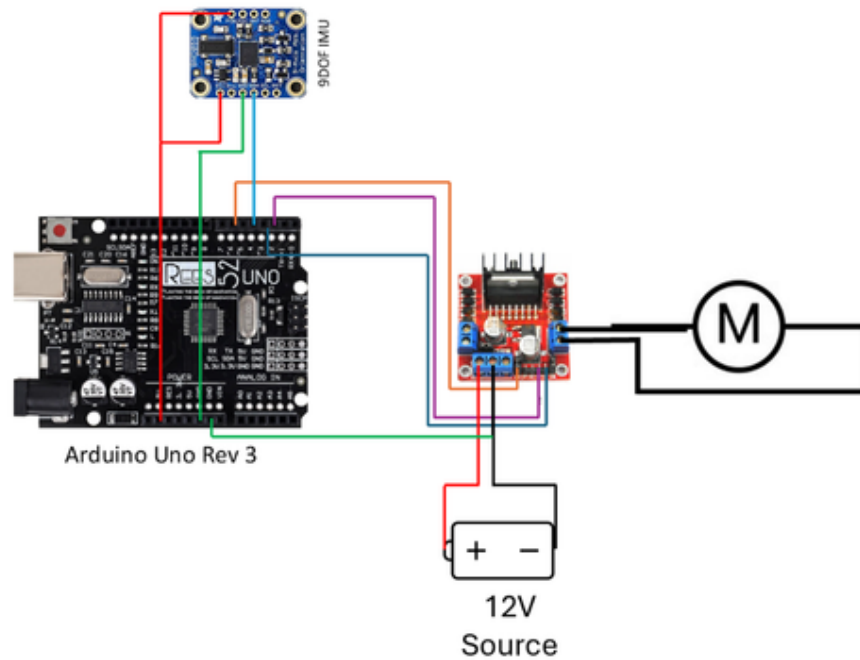
Electrical Integration and Wiring

The final electrical architecture was simple. The Arduino Uno R3 served as the central controller and was interfaced with the BNO085 IMU and the L298N motor driver. The BNO085 was used to provide the pendulum angle and angular velocity signals needed for feedback. The L298N was used to command the 12 V DC motor in both directions using digital logic inputs and an enable signal. The driver board accepts motor supply voltages from 5 V to 35 V and includes separate enable and direction-control inputs, making it suitable bidirectional DC motor control.

A 12 V battery pack was used as the main power source for the system. The battery powered the motor through the L298N driver, and the Arduino was also powered from the same battery supply. A common ground was established by connecting the battery ground to the H-bridge ground and then to the Arduino ground. This common reference was necessary to ensure that the logic signals sent from the Arduino to the motor driver were interpreted correctly. The final wiring schematic is included separately.

Sensor Integration and Calibration

The sensing strategy changed over the course of the project. The original plan was to use the AS5048A magnetic encoder to measure wheel speed or angular position. This encoder offers 14-bit angular resolution and can communicate through SPI or PWM, which would've been very easy to integrate into our project. However, the encoder was never fully integrated into the final working build. The main challenge was the difficulty of using the encoder to reliably track motor wheel speed in the physical setup, particularly while also completing the rest of the control implementation within the project schedule. As a result, the final controller did not rely on the encoder.



Circuit Diagram for the controller and sensor system

Fig. 18 Wiring schematic of the final electrical implementation.

Instead, the final implementation used the BNO085 IMU mounted directly to the pendulum arm. The IMU was attached to the arm so that it rotated with the body of the pendulum and could directly measure the states of interest. Sensor testing consisted of wiring the IMU to the Arduino, verifying communication, and checking whether the sensor outputs produced reasonable angle and angular velocity readings as the pendulum was moved manually. In the final implementation, pendulum angle and angular velocity were used directly from the IMU rather than from a combined encoder-IMU estimate. The BNO085 worked well because it integrates an accelerometer, gyroscope, and magnetometer with onboard sensor fusion and is capable of reporting angular velocity and orientation information.

To establish a reference angle, the pendulum was manually held in the upright configuration during initialization and that position was treated as zero degrees. This calibration procedure was straightforward and easy to implement, but it also introduced one of the known limitations of the experiment: any small misalignment while holding the arm upright at startup led directly to steady-state angle bias in the closed-loop response. This issue is consistent with the behavior discussed later in the controller evaluation section, where zero-position uncertainty contributed to steady-state error in the physical system.

Actuator Testing and Hardware Validation

Before closed-loop control was attempted, the motor and H-bridge were tested to verify basic operation. Initial actuator testing consisted of wiring the Greartisan 12 V motor to the L298N motor driver, commanding the motor in forward and reverse directions through the Arduino, and confirming that the motor could spin the reaction wheel in both directions. Once this basic functionality was established, the motor was mounted on the pendulum arm and tested with the reaction wheel attached. The main practical question was whether the actuator could generate enough reaction torque to drive the pendulum through the upright position.

These tests showed that the actuator was capable of influencing the pendulum motion, but they also exposed the main hardware limitations of the system. First, the selected DC motor was relatively heavy for the scale of the printed structure, which made the pendulum top-heavy and required the addition of a counterweight below the pivot. Second, although the motor could generate visible reaction torque, it lacked the quick reversal authority needed for more stabilization. Third, the use of an inexpensive geared DC motor and a simple L298N driver limited the available performance and contributed

a bad response when the arm crossed through the upright position. Nonetheless, the actuator testing confirmed that the hardware concept worked and that the motor-wheel combination could provide enough reaction torque to support controller development. The final hardware was therefore considered successful as an implementation platform, even though its limitations constrained the ultimate controller performance.

Overall, the final hardware implementation successfully demonstrated the core concept of a reaction-wheel inverted pendulum using 3D printed parts. The most important successes were the redesign of the pivot to reduce friction, the successful use of the BNO085 as the primary sensor, and the fabrication of a compact single-axis test article that could be integrated with the controller. The most important limitations were the heavy motor mass, top-heaviness of the arm, and the incomplete integration of the encoder. Even with these limitations, the final hardware provided a workable test for validating the control approach and demonstrating balancing behavior in the physical system.

B. Controller Implementation

The gains were experimentally tuned in the Arduino IDE and found to be:

$$K = \begin{bmatrix} 5.0 & 0.5 & 1.0 \end{bmatrix}. \quad (45)$$

These gains yielded a somewhat successful controlling of the system. There were a few issues with the gains chosen, the first was that the damping was too low and the arm would oscillate when it reached 0° . Normally, this would be resolved by increasing the K_d of the system, however any increase of K_d beyond 0.5 resulted in a deadband near 0° . It was assumed that this deadband was the result of motor jitter, a phenomena where the motor is being sent rapid contradictory signals which results in the motor freezing around 0° .

The next step to resolve this issue was the implementation of a lowpass filter to hopefully send the motor only lower frequency signals to prevent jitter. This was not able to be implemented because the lowpass filter could not successfully be tuned to achieve stability, the oscillatory reaction was too slow to achieve stability. The lowpass filter was then de-scoped. The following are actual responses given by the actual arm and control system during testing.

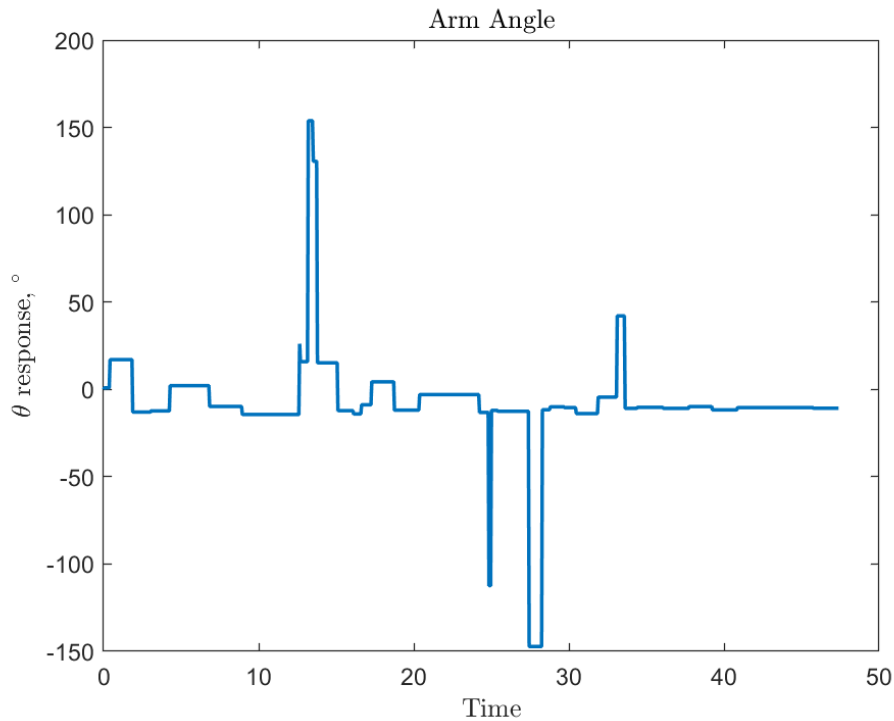


Fig. 19 Real controller reaction of the arm angle with respect to time.

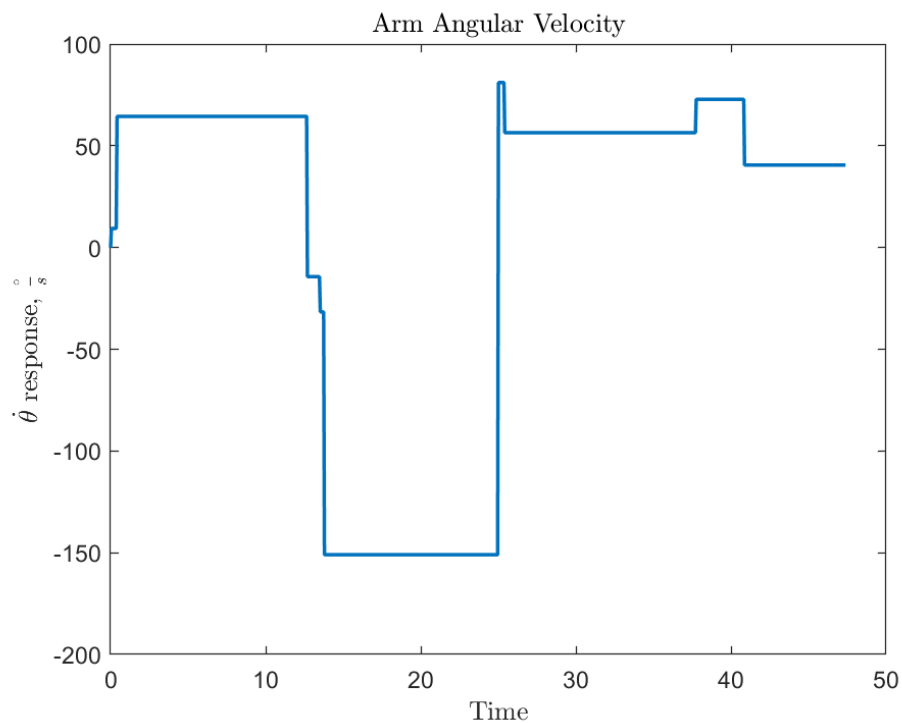


Fig. 20 Real controller reaction of the arm velocity with respect to time.

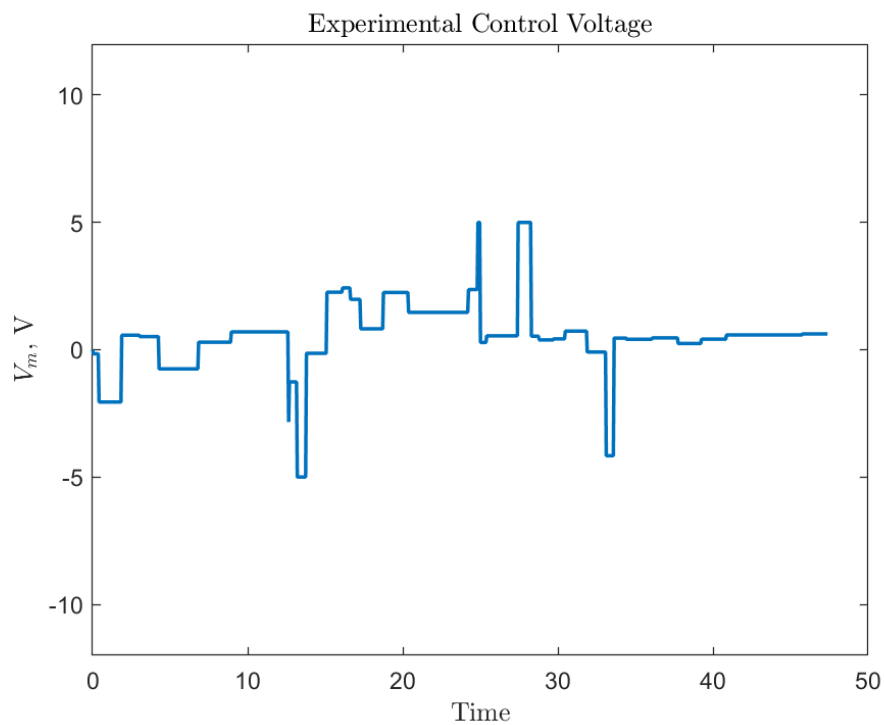


Fig. 21 Real controller reaction of the controller input voltage with respect to time.

There were perturbations to the system by physical impact at $t = 13$ seconds, 25, 27, and 32 seconds. The response by the velocity shows these as well, the velocity of the arm falling/coming back up correspond to the impacts. Lastly the control voltage shows the voltage going into the motor of the flywheel, it corresponds to the speed of the motor. It is showing constant response in reaction to the perturbations. It increases when the arm is hit hard, and increases at a lower magnitude when the arm is lightly tapped or undisturbed.

During this test, it can be seen that the final position of the arm does hover around about 10° , hence why it is somewhat successful. It is not within an acceptable error bound, though it did retain stability at it's chosen value. This shows that the arm did not have the ability to remain at 0° even with the motor on, possibly due to the deadband around the desired point.

In the future, if this project were to go on, we would need a stronger or lighter motor. The motor used right now cannot support its own weight with the power used and can not kick itself up from a fully fallen angle. A stronger motor would be able to provide a faster flywheel speed, creating enough angular momentum to properly control the mass of the system.

VII. Conclusion

Ultimately, the reaction wheel managed to successfully balance the inverted pendulum. The system was consistently able to return to its upright position by rotating the reaction wheel in the correct direction. It was able to balance in spite of impact forces applied to the top of the system. This means that the system behaved as expected. Unfortunately, however, none of the initial controller criteria was met. It was close, though. The measured specifications were:

- 1) $t_r = 0.05$ s, which is the only controller specification met by the system! The rise time is very low, indicating correct behavior based on the high integral gain.
- 2) $t_s = 30.48$ s, which is very high when compared to the set value of approximately two seconds. There are several possible reasons for the high settling time. There was significant oscillation in the system. Thus, more controller tuning – particularly adding damping – would help reduce the settling time. The damping was low in this system, as any increase in the derivative gain would cause the reaction wheel to stop spinning. It was not easily determinable the cause of this behavior.
- 3) $e_{ss} = 10.79^\circ$, which is quite large for a steady state error. The most likely reason lies in the implementation of the Arduino code. For the system to initialize, the pendulum must be held upright to discover the "zero" position. This could introduce significant error in where zero is relative to the upright. The extreme weight of the motor also skewed the center of mass and often caused the system to tip over.
- 4) $OS = 153.82^\circ$, which is a very large value. The most likely cause for this overshoot is the behavior of the system tending to slam to one side of the other. This problem was not able to be solved in the time given to complete this project, and the reason behind this behavior is unclear. It is believed that more controller tuning may be able to remedy this undesirable behavior.

So, while the controller specifications were not met, the behavior of the system was still somewhat expected. This gives a good ground point to continue further testing with reaction wheels and control the system properly.

The experimental data collected differed substantially from the simulated data. This demonstrates a potential error in the system modeling. A more rigorous model of the system that accounts for more realistic effects like air resistance and rotational friction might behave more closely to the measured behavior of the system. Or, perhaps, there were errors in modeling the simplified system that were missed by the authors. Another potential reason for the difference in data is an error in the use of the IMU. The data collected by the IMU tended to be inaccurate. This could be from the calibration as discussed previously, the Arduino code itself being errant, or from a misunderstanding of the wiring of the system. However, despite differing greatly, the data still follows a similar pattern. This is hopeful, as it demonstrates that the system behaves somewhat as expected. This leaves room for further iteration on the project.

For future repetition of this experiment, a few things should be considered for improvements. The biggest point would be the motor used. This experiment was conducted with a heavy motor that lacked power. A lighter, more powerful motor would work much better for an experiment like this. It would reduce the instability of the system (with the motor making the system top-heavy) and would allow for more torque produced by the reaction wheel. Another issue was the rotational friction that was present in the system. There were attempts to mitigate this by introducing ball bearings, sanding down parts of the system, and minimizing contact with the wood base, but friction still arose between some components. Continuing on reducing any friction points would improve the behavior of the system. Another point of improvement would certainly be with Arduino. None of the authors were familiar with the IDE, so the code is extremely raw and far from optimized. Adding someone to the team with extensive controlling experience would likely

improve the performance of the system as a whole.

References

- [1] ReM-RC, “Self balancing with reaction wheels (open source),” YouTube, Oct. 2022. URL <https://youtu.be/wacuNeE02zE>.